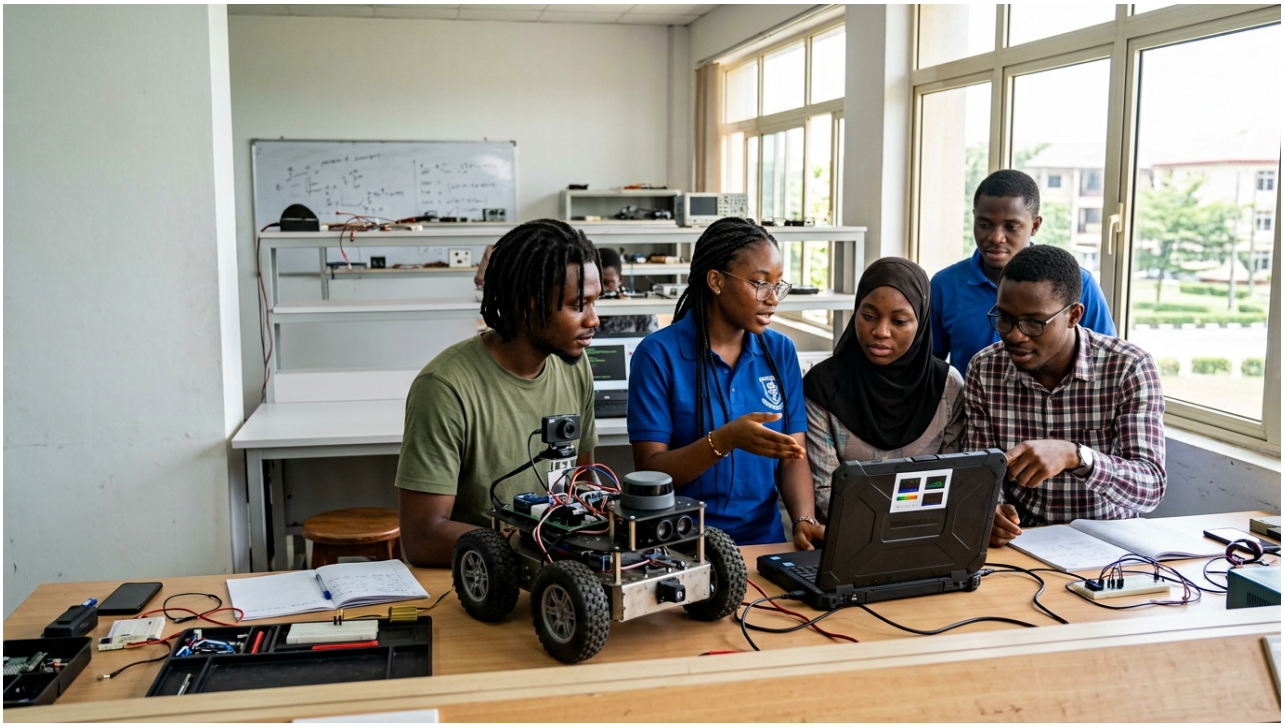
AI.101 — Introduction to Artificial Intelligence



Illustrative engineering-laboratory image; not documentary evidence of a specific university or event.

Narration manuscript · Module 6: Planning and Action

Academic level: Bachelor of Artificial Intelligence and Robotics Engineering
Content policy: No faith-related content.

1. From finding a path to choosing a sequence of actions

Search explored paths through a state space. Planning asks a related but more structured question: what sequence, or sometimes partially ordered set, of actions can transform the current situation into a state that satisfies a goal? A plan is more than a route. It may include tasks such as collecting an item, checking authorization, opening a door, moving to a destination, handing over a package, and recording completion.

Path search often treats actions as edges in a graph. Planning can reason about actions at a higher level by describing what must be true before an action can occur and what changes after it occurs. This helps an agent organize tasks whose relevant facts are not merely locations.

Consider a delivery robot. “Go to reception” is not a complete plan if the robot first has to acquire the package, confirm that the package identifier matches the request, and determine whether the delivery destination is staffed. The high-level task has dependencies. Movement is only one portion of the plan.

2. Actions: preconditions and effects

A simple planning action can be represented by a name, a set of **preconditions**, and a set of **effects**. Preconditions describe facts that must hold for the action to be applicable. Effects describe facts that become true or false when the action succeeds.

For example, `Move(Robot, CorridorA, CorridorB)` may require that the robot is at CorridorA, the passage is open, the robot has sufficient battery, and the intended corridor segment is safe to enter. Its effects may include that the robot is now located at CorridorB and that battery charge has decreased slightly.

A `PickUp(Package)` action may require that the robot is at the package location, the package is identified correctly, and the gripper is available. Its effects may include that the package is secured and the gripper is occupied. A `Deliver(Package, Reception)` action may require that the robot is at reception, the package remains secured, and an authorized recipient is present. Its effects may include that the package is handed over and delivery is recorded.

This representation is simplified. Real actions may take time, fail, have uncertain effects, or require continuous control. Nevertheless, preconditions and effects help us detect impossible plans. If the plan includes delivering a package before picking it up, the action conditions reveal the error.

Show action cards with preconditions above a horizontal divider and effects below. Animate facts from the current state satisfying the conditions; then highlight which state facts change after the action.

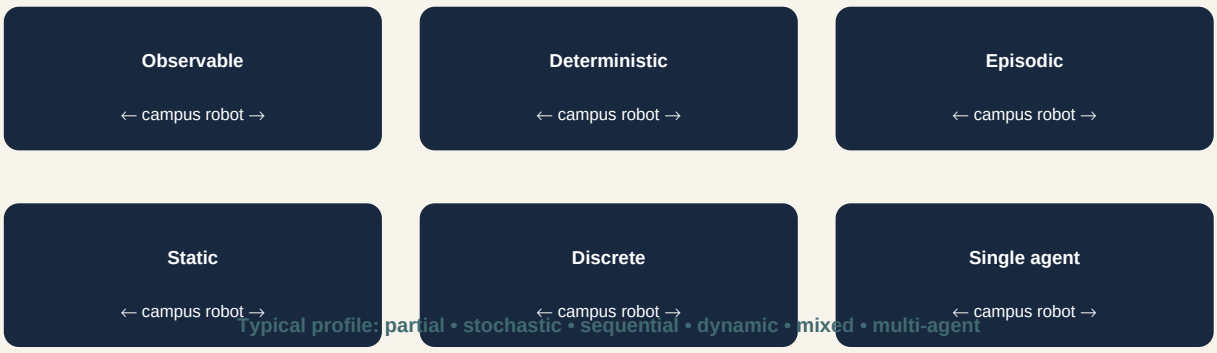


Diagram: Environment classification profile

3. Forward planning and backward planning

A forward planner begins with the current state and applies actions that are currently possible. Each action creates a successor state. The planner continues until it finds a state that satisfies the goal. This resembles state-space search and is intuitive when the initial state is well defined and the action set is manageable.

A backward planner begins with the goal and asks which action could make the goal true. It then asks what preconditions that action requires, treating those preconditions as subgoals. For the delivery example, the goal “package delivered to reception” suggests an action that requires the robot at reception and the package secured. The robot-at-reception subgoal may require a navigation plan; the package-secured subgoal may require a pickup action. Backward reasoning can focus the search on actions relevant to the goal.

Neither direction is always superior. Forward planning can generate many irrelevant possibilities before reaching a distant goal. Backward planning can become complicated when many actions can achieve a subgoal or when preconditions interact. Some systems combine strategies or use heuristics to guide the search.

4. A simple planning example

Let the initial state contain these facts: robot at laboratory, package at laboratory, package not secured, route to reception available, and reception open. The goal is package delivered and delivery recorded.

A possible plan is: verify package identity; secure package; navigate to reception; confirm recipient; hand over package; record delivery. The exact actions depend on the robot’s capabilities and operating procedures. If the robot has no barcode reader, “verify package identity” may require an operator. If the reception desk is closed, the plan needs a wait, a new destination, or an assistance request.

We can inspect action dependencies. “Secure package” must happen before “navigate with package,” because the package must be held. “Confirm recipient” must occur before “hand over package.” “Record delivery” must occur after the handover. Navigating and checking a remote schedule may be parallelizable if the system can perform both concurrently, but only if the schedule information is trustworthy and current.

The plan should also represent negative conditions. If the robot must not enter a restricted corridor, it is not enough to record the desired route. The planner or a separate safety layer must enforce the prohibition. A plan can be logically valid in a simplified model and still be unacceptable in the actual environment if constraints are omitted.

5. Total-order and partial-order plans

A **total-order plan** specifies a complete sequence: action one, then action two, then action three. It is simple to understand and execute, but may impose unnecessary ordering. If the robot can verify a recipient’s availability while navigating, a strict total order might unnecessarily delay the task.

A **partial-order plan** specifies only the ordering constraints that are required. For example, package identification must precede pickup; pickup must precede carrying; arrival must precede handover; handover must precede completion logging. Other compatible actions may occur in parallel. Partial ordering can make plans more efficient and flexible, but it requires execution machinery that can manage concurrent actions and resolve conflicts.

In robotics, concurrency requires care. Two software tasks can run in parallel while competing for the same sensor, processor, network connection or physical resource. The planner must know when actions interfere. Moving the base while manipulating a fragile object may be unsafe even if the symbolic preconditions appear satisfied. Resource constraints, timing and physical feasibility must be included at an appropriate level.

6. Planning versus reactive control

Planning and reactive control operate at different horizons. A planner reasons about what sequence of tasks or route segments may achieve a goal. A controller makes frequent adjustments based on sensor feedback. A mobile robot may plan to move through the north corridor, while its controller continuously regulates wheel speeds and maintains a safe distance from nearby obstacles.

A plan is not a guarantee that the world will follow predictions. An obstacle can appear; a door can close; a wheel can slip; a person can ask the robot to stop. The system must compare expected state changes with observations. This process is called **execution monitoring**. When a mismatch is detected, the agent may update its state estimate, pause, repair the current plan, replan from the new state or request assistance.

Three linked layers—task planner, route/motion planner, feedback controller—with a safety monitor beside them. Show observations returning to the controller and state updates returning to the planner.

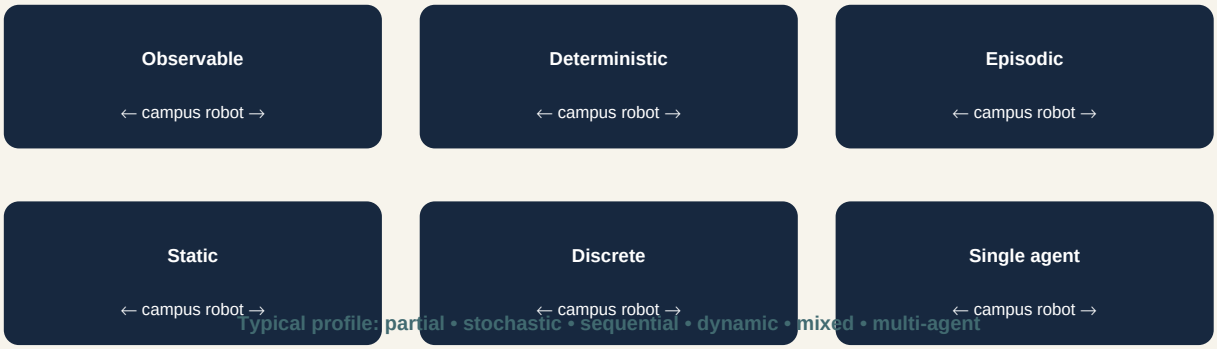


Diagram: Environment classification profile

Reactive behavior can be useful when immediate response matters. A proximity sensor may trigger braking without waiting for a global planner to recompute an entire mission. But purely reactive behavior can produce loops or short-sighted choices. Planning provides longer-term structure; feedback provides adaptation. A robust robotic system coordinates the two.

7. Replanning after a change

Suppose the robot has planned a route through Corridor B. During execution, a sensor indicates that a door has closed. The original plan is no longer executable. The agent should not keep issuing movement commands based on a stale plan. It must update the relevant state, determine whether an alternate route exists, and check the consequences for time and battery.

Replanning can start from the current estimated state and solve the full problem again. Alternatively, the system may repair only the affected section. Full replanning is simpler conceptually but may take more computation. Local repair can be faster but risks preserving an assumption that has become invalid elsewhere. The appropriate strategy depends on task scale, time limits and the reliability of the model.

The system should also decide when to ask for human help. If a route is blocked and no safe alternative is known, repeatedly searching the same map will not solve the problem. A useful autonomy design recognizes the boundary of its competence and hands off with enough context for a person to help: current location estimate, observations, attempted actions, and reason for stopping.

8. Practical lab: plan, execute, monitor

Build a small simulated delivery task with a robot, package, pickup location, destination, doors and a charging station. Represent actions using preconditions and effects. At minimum, include move, pick up, deliver, wait, charge and request assistance. Add constraints for restricted zones and low battery.

First, generate a total-order plan from a clean initial state. Then alter the state: close a door, reduce battery charge, or make the recipient unavailable. Execute the first few actions in a simple simulator or scripted environment. Compare the predicted state with the observed state after each action. When they differ, log the discrepancy and decide whether to repair, replan, stop or request assistance.

Test at least one failure case in which an action does not have its expected effect. For instance, the pickup action may fail because the package is misaligned. The planner must not mark the package secured merely because it issued the command. It should require a confirming observation, such as a gripper sensor or visual check.

For an extension, compare forward planning with backward planning on the same task. Measure the number of states or partial plans explored. Explain how the goal structure and action descriptions affect performance. Report limitations honestly: a small simulation demonstrates logic, not physical safety or deployment readiness.

9. Common planning mistakes

One mistake is to confuse a plan with a script of commands. A plan is conditional on a model and may need monitoring. A command sequence that ignores feedback is brittle.

Another mistake is to omit action preconditions. A planner may schedule pickup while the robot is across the building or handover when no recipient is present.

A third mistake is to record only positive effects. Actions can also consume resources, occupy tools, close doors or make other actions unavailable. Omitting these changes produces plans that appear feasible but cannot be executed.

A fourth mistake is to replan from an assumed state rather than the best current estimate. After a failed action, the robot may not be where the planner expected. State estimation and plan repair must be coordinated.

A fifth mistake is to treat safety as a final filter after planning. Safety constraints should shape the action model and execution architecture from the beginning. A planner should not routinely generate unsafe plans and rely on a last-second stop mechanism as its only protection.

10. Knowledge check and assignment

What is the difference between a precondition and an effect? How does backward planning work from a goal? What does a partial-order plan permit that a total-order plan does not? Why must an agent monitor execution? What should happen when an action is issued but its expected effect is not observed? How can a planner hand off safely to a human operator?

For the assignment, implement the simulated delivery task with action preconditions, effects, goals and resource constraints. Demonstrate a normal plan, a failed action and a changed environment. Include an execution trace showing predicted and observed states. Submit a short analysis of plan validity, safety assumptions, replanning behavior and one case where the agent correctly requests assistance rather than continuing autonomously.

Module summary

Planning organizes actions to achieve goals. We represented actions with preconditions and effects, compared forward and backward planning, examined total-order and partial-order plans, and distinguished planning from reactive control. We then added execution monitoring and replanning so that agents can respond when the world differs from their model. The central principle is that a plan is a reasoned proposal for action, not a promise that execution will unfold exactly as expected.